



Hochschule
Bonn-Rhein-Sieg
University of Applied Sciences

Introduction to Python Programming

Foundation Course @ Hochschule Bonn-Rhein-Sieg (H-BRS)

By Rahul Bale

SEPT 2026

Course Curriculum Roadmap

1. Introduction to Python
2. Why Learn Python?
3. Getting started with Python
4. Environment & Setup
5. Core Syntax & Data Types
6. Operators & Mathematical Expressions
7. Control Flow & Pattern Matching
8. Core Collections: Lists, Tuples, Dictionaries & Sets
9. Functions
10. File I/O & Exceptions
11. Modules and the standard library
12. File Handling
13. Error Handling & Exceptions
14. Object-oriented programming in Python
15. Python Data Model & Dunder Methods
16. What's next in OOP?
17. Inheritance, Polymorphism, Encapsulation, Abstractions

Introduction to Python

Python is a high-level, interpreted programming language known for its simplicity and readability.

1. **High-level:** language abstracts away most of the complex details of the computer hardware
2. **Interpreted:** code is executed “directly”, line-by-line, by an interpreter

Why Learn Python?

- **Versatile:** Used in web development, data science, automation, and more.
- **Easy to Learn:** Simple syntax, great for beginners.
- **For us, this is important because:**
 - **Almost all machine learning libraries are developed in Python**
 - **For robotics, it interfaces well with ROS2**
 - **Many mathematics problems are best solved with a language like Python**

Getting Started with Python

- **Installing Python: Download from python.org. more.**
- **Python 3.x is recommended.**
- **Writing Python code in:**
 - **Interactive Mode: Python Shell**
 - **Script Mode: Using a text editor or IDE like VS Code, PyCharm.**

Setting Up the Python Development Environment

- **Virtual environments isolate dependencies per project and avoid polluting the global OS Python.**
- **Interactive notebooks vs. standalone script entry points.**



```
# 1. Create a dedicated virtual environment
$ python3 -m venv .venv

# 2. Activate the virtual environment
$ source .venv/bin/activate      # Linux / macOS
$ .venv\Scripts\activate        # Windows PowerShell

# 3. Verify active python binary & version
$ which python3 && python --version
# -> /path/to/project/.venv/bin/python3

# 4. Launch interactive shell or script
$ python3
>>> print('Environment isolated!')
>>> exit()

# 5. Run your script
$ python3 main.py
```

Basic Syntax

- Python scripts typically end with `.py`.
- Python uses indentation to define code blocks.
- As the tradition goes, here's our first Python code:



```
print("Hello world!").
```

```
if True:  
    print("This is true.")
```

```
# Comments start with #. This is the comment
```

```
"""
```

```
Multi line commets
```

```
you can write multile line comments without using #
```

```
"""
```

Variables, Types & Formatting

These variables are examples of the basic data types:

1. **Integer:** whole number without a decimal point (age = 20)
2. **Float:** numbers with decimal point (temperature = 18.2)
3. **String:** sequence of characters in single or double quotes (name = 'Alice')
4. **Boolean:** represents a binary true/false value (studies_at_hbrs = True)

String Formatting: `print(f"Reading: {sensor_id=}, range={distance_m:.2f} m, status={is_active}")`

Operators & Mathematical Expressions

- **Arithmetic Operators:**

- Addition: +
- Subtraction: -
- Multiplication: *
- Division: /

- **Comparison operators:**

- Equal to: ==
- Not equal to: !=
- Greater than: >
- Less than: <

- **Logical operators:** `and`, `or`, `not` (short-circuiting evaluation).

```
sum = 5 + 3
difference = 5 - 3
product = 4 * 3
quotient = 20 / 4

# Division differences
div_float: float = 17 / 4      # 4.25 (standard float division)
div_floor: int = 17 // 4      # 4 (integer floor division)
remainder: int = 17 % 4      # 1 (remainder of the division)
power_val: int = 2 ** 8      # 256 (exponentiation)

# Identity vs Value Equality
list_a: list[int] = [1, 2, 3]
list_b: list[int] = [1, 2, 3]

print(list_a == list_b)      # True (same contents)
print(list_a is list_b)      # False (distinct memory objects!)
```

Control Flow (Conditional Statements)

- **Conditional branching** with ``if`, `elif`, and `else`` blocks governed by indentation.
- **Iteration constructs:**
 - **For Loop:** Iterating over a sequence (e.g., list, string).
 - **While Loop:** Continues as long as a condition is true.

```
● ● ●

# if-elif-else statements:
a = 20
b = 30

if a > b:
    print("a is greater than b")
elif a == b:
    print("a and b are equal")
elif a >= 20:
    print("a is greater than equal to 20")
else:
    print("b is greater than a")

# for loop
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print("fruits: ", fruit)

# while loop
count = 0
while count < 5:
    print("count: ", count)
    count += 1
```

Control Flow

Control flow cont.: As part of control flow statements, you should be familiar with break and continue keywords.



```
# BREAK statement
# Use a for loop to find and print the first positive number in a list

numbers = [-3, -2, -1, 0, 1, 2, 3]
for number in numbers:
    if number > 0:
        print("The first positive number is: ", number)
        break # Exits the loop

# CONTINUE statement
# Print only odd numbers from 0 to 10
for number in range(11):
    if number % 2 == 0:
        continue # Skips the rest of the loop and continues with the next iteration
    print("Odd number:", number)
```

Core Collections: Lists, Tuples, Dictionaries & Sets

- **Lists (list):** Mutable, ordered sequence; indexing, slicing, dynamic growth written with square brackets [].
- **Tuples (tuple):** An ordered, unchangeable sequence used to protect fixed data. with rounded brackets ().
- **Dictionaries (dict):** Key-value hash maps maintaining insertion order. written with curly brackets {}.
- **Sets (set):** An unordered, mutable collection of unique elements written with curly brackets {}.



```
List: fruits = ["apple", "banana", "cherry"]
```

```
Tuple: coordinates = (40.7128, -74.0060)
```

```
Dictionary: user = {"name": "Alex", "age": 25}
```

```
Set: unique_ids = {101, 102, 103}
```

Data Transformations

- List, dict, and set comprehensions provide declarative, readable collection transformations.
- Comprehensions are faster than manual `.append()` accumulator loops due to C-level bytecode optimization.
- Use `enumerate()` for clean index-value tracking instead of manual counter variables (`i += 1`).
- Use `zip()` to iterate through parallel sequences cleanly without index tracking.
- Dictionary access and iteration is done using `.items()`

```
# 1. List & Dictionary Comprehensions
raw_depths = [1.25, -0.05, 3.42, 0.88, -1.0]
valid_depths = [d for d in raw_depths if d > 0.0]
squares = [x**2 for x in range(1, 11)]

# 2. Enumerate: index + element
for idx, val in enumerate(valid_depths):
    print(f"Sample {idx}: {val:.2f} m")

# 3. Zip: pair multiple iterables
sensor_names = ["front", "rear", "top"]
health_status = [True, True, False]
for name, status in zip(sensor_names, health_status):
    state = "ONLINE" if status else "FAULT"
    print(f"{name.upper()}: {state}")

# Dictionary access and iteration
for key, value in user.items():
    print(f"{key}, {value}")
```

Function

Function is a block of reusable code that performs a specific task.

Why would you use them?

1. Organize code into manageable sections
1. Improve code readability
2. Facilitate code reuse
3. Reduce redundancy



```
# Simple function
def greet():
    print("Hello, welcome to Python programming!")
greet()

#Function with parameters and a return statement
def add_numbers (number1, number2):
    sum = number1 + number2
    return sum

result = add_numbers (5, 3)
print("The sum is:", result)

#Function with default function arguments
def greet (name="User"):
    print("Hello,", name + "! Welcome to Python programming.")

greet ("Alice")
greet()
```

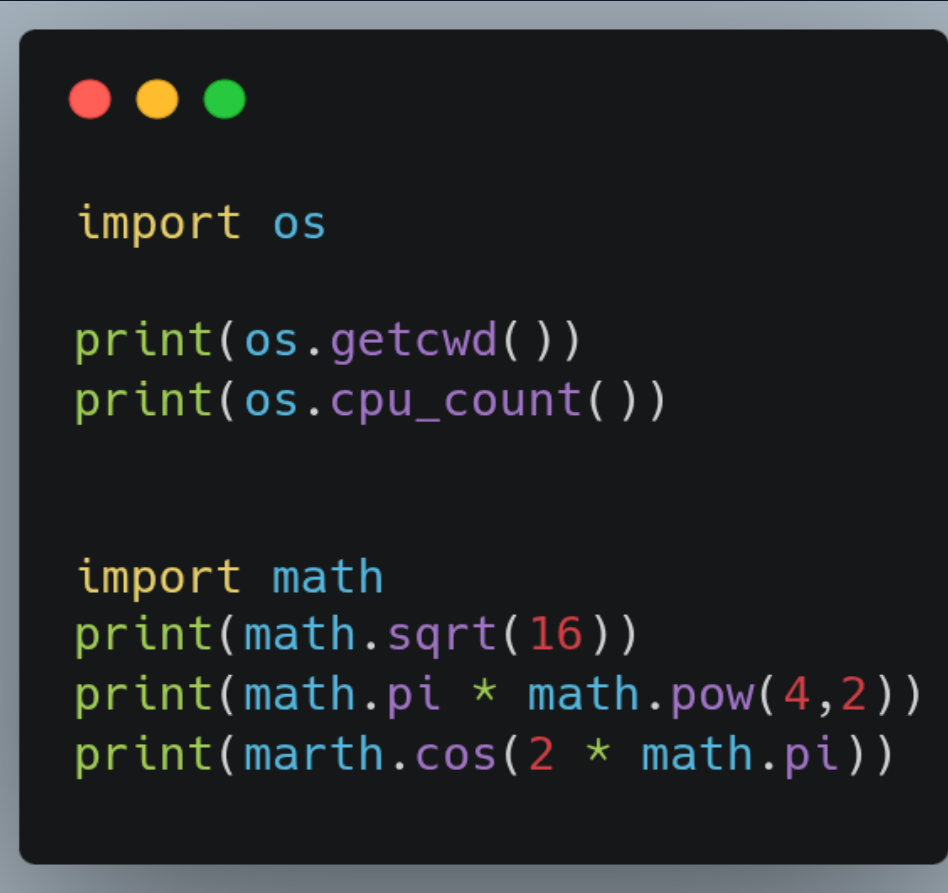
Modules and the standard library

Modules:

1. Python file containing definitions/functions/statements
2. Designed to organize Python code for better readability and reusability
3. Can be imported, and divided into separate namespaces
4. Python standard library is a collection of modules included with Python.

Common Libraries:

1. os: Interact with the operating system.
2. sys: System-specific parameters and functions.
3. datetime: Manipulate dates and times.
4. random: Generate random numbers.



```
import os

print(os.getcwd())
print(os.cpu_count())

import math
print(math.sqrt(16))
print(math.pi * math.pow(4, 2))
print(marth.cos(2 * math.pi))
```

File Handling

I/O (Input/Output) operations allow programs to interact with the file system, networks, and other devices.

```
from pathlib import Path
import json

# Opening a File
with open("example.txt", 'r') as file:
    data = file.read()

# Writing to a file
with open("example.txt", 'a') as file:
    file.write("Hello from python! \n")

# Modern path manipulation
file_path = Path("experiment_log.json")

# 1. Writing structured JSON safely
data = {'experiment': 'ToF_Scan', 'samples': [1.2, 3.4, 5.6]}
with open(file_path, mode='w', encoding='utf-8') as f:
    json.dump(data, f)

# 2. Reading back the data
with open(file_path, mode='r', encoding='utf-8') as f:
    loaded = json.load(f)
print('Loaded Experiment:', loaded['experiment'])
# -> Loaded Experiment: ToF_Scan
```

Error Handling & Exceptions

- Exception get “raised” when there's a irreconcilable problem in the code.
- Exception handling is a mechanism for gracefully responding/recovering from errors during program's execution.



```
# 1. Define a custom error
class SensorError(Exception):
    pass

# 2. Function that checks a value
def read_temp(voltage):
    if voltage > 5.0:
        raise SensorError("Voltage too high!")
    return voltage * 25

# 3. Handle the error cleanly
try:
    temp = read_temp(6.0)
except SensorError as err:
    print("Error:", err)
else:
    print("Temp:", temp)
finally:
    print("Done checking.")
```

Object-oriented programming in Python (Part 1)

Object-oriented programming (OOP) is a paradigm based on the concept of "objects". These objects can contain data (**attributes**) and instructions (**methods**).

Important to know the difference between a class and an object:

- Class: **blueprint** for creating objects, they define attributes and methods
- Object: **instance** of a class

Importance of OOP

- Importance of OOP Many of your assignments will assume basic understanding of OOP.
- Moreover, you will be working with ROS2 a lot. Creating subscriber/publisher nodes will demand understanding of OOP.

Object-oriented programming in Python (Part 2)

```
class Robot:
    """Represents a mobile robot with a name and battery."""

    # __init__(): The constructor method that initializes instance attributes when an object is
    # created.
    # self parameter: An explicit reference to the current object instance accessing its own data.
    def __init__(self, name: str, battery_pct: int = 100):
        # Instance attributes
        self.name: str = name
        self.battery_pct: int = battery_pct

    # Methods in a Class
    # Methods are functions defined inside a class that operate on the object's data.
    def drive(self, distance_m: float) -> None:
        """Simulates driving and depleting battery."""
        self.battery_pct -= int(distance_m * 2)
        print(f'{self.name} drove {distance_m}m. Battery: {self.battery_pct}%')

# creating an object
bot = Robot(name='TurtleBot', battery_pct=95)
bot.drive(10.0)
# -> TurtleBot drove 10.0m. Battery: 75%
```

What's next in OOP?

If you are coming from a computer science background, you will most likely be familiar with the following four pillars of OOP:

1. **Inheritance:** Inheritance allows a new class to adopt the attributes and methods of an existing class, saving time and preventing repeated code.
2. **Abstractions:** Showing only what is necessary to the user and hiding the complicated internal machinery.
3. **Encapsulation:** Encapsulation refers to the bundling of data (attributes) and methods that operate on the data within a class. It restricts direct access to some of the object's components. Follow the convention by prefixing the name of the member by a single underscore "_".
4. **Polymorphism:** Polymorphism means different classes can use the exact same method name, but each one carries it out in its own distinct way.

If you're not, don't worry: learn by building!

Inheritance, Polymorphism, Encapsulation, Abstractions



```
# 1. Base Class (Abstraction: provides a simple blueprint)
class Animal:
    def __init__(self, name):
        self._name = name # Encapsulation: hidden/protected data

    def speak(self):
        pass # Abstraction: hides details, child classes fill it in

# 2. Inheritance: Dog and Cat reuse Animal's code
class Dog(Animal):
    def speak(self): # Polymorphism: same method name, different action
        return f"{self._name} says Woof!"

class Cat(Animal):
    def speak(self): # Polymorphism: same method name, different action
        return f"{self._name} says Meow! and the \
first letter of cat is {self._name[0]}"

# In Action:
pets = [Dog("Buddy"), Cat("Luna")]
for pet in pets:
    print(pet.speak())
```

Thank you!

Any questions?